

Atividade Avaliativa - Python

Disciplina: Computational Thinking using Python **Prof^a.** Klébia Thomé

Integrantes

- Matheus Ferreira Antônio
- João Vitor Cruz de Lima

RM

- 570933
 - 571277
-

1. Explicação resumida

O programa implementa um sistema de controle de estoque em memória, utilizando uma **lista de dicionários** (produtos), onde cada dicionário representa um produto com nome, preço, quantidade e categoria.

O menu principal oferece as opções obrigatórias (listar, cadastrar, buscar, analisar estoque, ordenar por preço e sair) além das duas opções do desafio extra (remover produto pelo nome e identificar a categoria com maior valor total em estoque).

Uma **tupla** (`categorias_validas`) é usada para restringir o cadastro a um conjunto fixo de categorias — a imutabilidade da tupla é adequada porque essa lista de opções não deve ser alterada durante a execução do programa.

Todas as entradas do usuário passam por validação com `try/except` e `raise ValueError`, cobrindo os casos exigidos: preço/quantidade em formato inválido, preço menor ou igual a zero, quantidade negativa, campos obrigatórios vazios, opção de menu inexistente (tratado com `case _ no match`) e produto não encontrado na busca/remoção. Em nenhum desses casos o programa é encerrado — ele sempre retorna ao menu.

2. Evidências de execução

2.1 Listagem de produtos

```
=====KABUM=====
1 - Listar produtos
2 - Cadastrar produtos
3 - Buscar produtos
4 - exibir análise de estoque
5 - Ordenar Produtos
6 - Remover Produtos
7 - Categoria de Maior Valor
8 - Sair

->
=====LISTA DE PRODUTOS=====
Nome : Notebook|Preço: R$4500.00|Quantidade: 10|Categoria:
```

Informática

Nome : Mouse|Preço: R\$99.99|Quantidade: 2|Categoria: Informática

Nome : Teclado|Preço: R\$359.50|Quantidade: 20|Categoria: Informática

Nome : Tablet|Preço: R\$2899.99|Quantidade: 25|Categoria: Informática

Nome : Headset|Preço: R\$200.50|Quantidade: 20|Categoria: Informática

=====KABUM=====

- 1 - Listar produtos
- 2 - Cadastrar produtos
- 3 - Buscar produtos
- 4 - exibir análise de estoque
- 5 - Ordenar Produtos
- 6 - Remover Produtos
- 7 - Categoria de Maior Valor
- 8 - Sair

->

Encerrando sistema...

2.2 Cadastro de produto

=====KABUM=====

- 1 - Listar produtos
- 2 - Cadastrar produtos
- 3 - Buscar produtos
- 4 - exibir análise de estoque
- 5 - Ordenar Produtos
- 6 - Remover Produtos
- 7 - Categoria de Maior Valor
- 8 - Sair

->

=====CADASTRO DE PRODUTOS=====

Digite um nome válido - > Digite um preço válido - > Digite uma
quantidade válida - > Digite uma categoria válida - >

Produto 'Monitor gamer' cadastrado com sucesso!

=====KABUM=====

- 1 - Listar produtos
- 2 - Cadastrar produtos
- 3 - Buscar produtos
- 4 - exibir análise de estoque
- 5 - Ordenar Produtos
- 6 - Remover Produtos
- 7 - Categoria de Maior Valor
- 8 - Sair

->

=====LISTA DE PRODUTOS=====

Nome : Notebook|Preço: R\$4500.00|Quantidade: 10|Categoria:
Informática

Nome : Mouse|Preço: R\$99.99|Quantidade: 2|Categoria: Informática

Nome : Teclado|Preço: R\$359.50|Quantidade: 20|Categoria: Informática

Nome : Tablet|Preço: R\$2899.99|Quantidade: 25|Categoria: Informática

Nome : Headset|Preço: R\$200.50|Quantidade: 20|Categoria: Informática

Nome : Monitor gamer|Preço: R\$1299.90|Quantidade: 8|Categoria:
Informática

=====KABUM=====

- 1 - Listar produtos
- 2 - Cadastrar produtos
- 3 - Buscar produtos
- 4 - exibir análise de estoque
- 5 - Ordenar Produtos
- 6 - Remover Produtos
- 7 - Categoria de Maior Valor
- 8 - Sair

```
->
Encerrando sistema...
```

2.3 Busca de produto

```
=====KABUM=====
1 - Listar produtos
2 - Cadastrar produtos
3 - Buscar produtos
4 - exibir análise de estoque
5 - Ordenar Produtos
6 - Remover Produtos
7 - Categoria de Maior Valor
8 - Sair

->
=====BUSCA DE PRODUTOS PELO NOME=====
Digite o nome do produto que deseja buscar: Preço:
R$359.50|Quantidade: 20|Categoria: Informática
```

```
=====KABUM=====
1 - Listar produtos
2 - Cadastrar produtos
3 - Buscar produtos
4 - exibir análise de estoque
5 - Ordenar Produtos
6 - Remover Produtos
7 - Categoria de Maior Valor
8 - Sair
```

```
->
Encerrando sistema...
```

2.4 Tratamento de erro (preço inválido)

```
=====KABUM=====
1 - Listar produtos
2 - Cadastrar produtos
3 - Buscar produtos
4 - exibir análise de estoque
5 - Ordenar Produtos
6 - Remover Produtos
7 - Categoria de Maior Valor
8 - Sair

->
=====CADASTRO DE PRODUTOS=====
Digite um nome válido - > Digite um preço válido - > Erro: could not
convert string to float: 'abc'. Cadastro cancelado, tente novamente.
```

```
=====KABUM=====
1 - Listar produtos
2 - Cadastrar produtos
3 - Buscar produtos
4 - exibir análise de estoque
5 - Ordenar Produtos
6 - Remover Produtos
7 - Categoria de Maior Valor
8 - Sair
```

```
->
Encerrando sistema...
```

3. Código-fonte completo

#Lista de dicionários. Contém 5 produtos e suas informações, por isso a perfeição que é dicionários em python

```
produtos = [
    {
        "nome": "Notebook",
        "preço": 4500,
        "quantidade": 10,
        "categoria": "Informática"
    },
    {
        "nome": "Mouse",
        "preço": 99.99,
        "quantidade": 2,
        "categoria": "Informática"
    },
    {
        "nome": "Teclado",
        "preço": 359.50,
        "quantidade": 20,
        "categoria": "Informática"
    },
    {
        "nome": "Tablet",
        "preço": 2899.99,
        "quantidade": 25,
        "categoria": "Informática"
    },
    {
        "nome": "Headset",
        "preço": 200.50,
        "quantidade": 20,
        "categoria": "Informática"
    }
]

# Tupla com as categorias aceitas no cadastro. Usamos tupla (e não
lista) porque
# esse conjunto de categorias válidas é fixo durante a execução do
programa
# e não deve ser alterado – a imutabilidade da tupla protege esses
dados.
categorias_validas = ("Informática", "Casa", "Brinquedos")

#Criamos uma função que trará uma repetição que para cada item
dentro da lista, ele vai pegar as informações contidas dentro dela e
vai imprimir no terminal
def listar_produto():
    print("\n=====LISTA DE PRODUTOS=====")
    for produto in produtos:
        print(f"Nome : {produto['nome']}|Preço:
R${produto['preço']:.2f}|Quantidade:
{produto['quantidade']}|Categoria: {produto['categoria']}")

#Criando a função de cadastrar produto

def cadastrar_produto():
    print("\n=====CADASTRO DE
PRODUTOS=====")
    try:
        novo_nome = input("Digite um nome válido - >
").strip().lower().capitalize()
        if novo_nome == "":
            raise ValueError("O nome não pode ser vazio")

        novo_preco = float(input("Digite um preço válido - > "))
```

```

        if novo_preco <= 0:
            raise ValueError("Preço não pode ser 0 ou menos")

        nova_quantidade = int(input("Digite uma quantidade válida -
> "))

        if nova_quantidade < 0:
            raise ValueError("Não pode se ter um estoque negativo")

        nova_categoria = input("Digite uma categoria válida - >
").strip().lower().capitalize()
        if nova_categoria == "":
            raise ValueError("A categoria não pode ser vazia")
        if nova_categoria not in categorias_validas:
            raise ValueError(f"Categoria inválida. Use uma
das opções: {categorias_validas}")

        novo_produto = {
            "nome": novo_nome,
            "preço": novo_preco,
            "quantidade": nova_quantidade,
            "categoria": nova_categoria
        }

        produtos.append(novo_produto)
        print(f"\nProduto '{novo_nome}' cadastrado com sucesso!")

    except ValueError as erro:
        print(f"Erro: {erro}. Cadastro cancelado, tente novamente.")

#Criamos a função que irá percorrer uma lista para encontrar um
produto, criando uma variável que receberá o nome que a pessoa
deseja buscar e uma variável que dirá se encontramos, ou não, e será
um valor booleano False. Fazemos uma estrutura de repetição For In,
para percorrer a lista com um If, para caso o item da lista seja
igual ao nome que a pessoa quer buscar. Se for, ela imprime o preço,
quantidade e categoria do produto, e muda a variável encontrado para
True. Por fim, o If not encontrado fora da repetição para que não
tenha varias das mesmas mensagens negativas caso não seja encontrado
o produto na lista
    def buscar_produto():
        print("\n=====BUSCA DE PRODUTOS PELO NOME=====")
        buscar_nome = input("Digite o nome do produto que deseja buscar:
").strip().lower().capitalize()
        encontrado = False
        for produto in produtos:
            if produto["nome"] == buscar_nome:
                print(f"Preço: R${produto['preço']:.2f}|Quantidade:
{produto['quantidade']}|Categoria: {produto['categoria']}")
                encontrado = True
        if not encontrado:
            print("Produto não encontrado")

    def analisar_estoque():
        print("\n=====ANALISE DO ESTOQUE=====")
        if len(produtos) == 0:
            print("Nenhum produto cadastrado para analise.")
            return
        quantidade_produtos = len(produtos)
        valor_total = 0
        quantidade_total_itens = 0

        produto_maior_valor = produtos[0]
        produto_menor_quantidade = produtos[0]

        for produto in produtos:
            valor_total += produto["preço"] * produto["quantidade"]
            quantidade_total_itens += produto["quantidade"]
            if produto["preço"] > produto_maior_valor["preço"]:

```

```

        produto_maior_valor = produto
        if produto["quantidade"] <
produto_menor_quantidade["quantidade"]:
            produto_menor_quantidade = produto

    print(f"Quantidade de produtos cadastrados:
{quantidade_produtos}")
    print(f"Valor total do estoque: R${valor_total:.2f}")
    print(f"Produto com maior preço: {produto_maior_valor['nome']}
(R${produto_maior_valor['preço']:.2f})")
    print(f"Produto com menor quantidade:
{produto_menor_quantidade['nome']}
({produto_menor_quantidade['quantidade']} unidades)")
    print(f"Quantidade total de itens em estoque:
{quantidade_total_itens}")

def ordenar_produto():
    print("\n=====PRODUTOS ORDENADOS POR PREÇO=====")
    if len(produtos) == 0:
        print("Nenhum produto cadastrado para análise.")
        return

    produtos_ordenados = sorted(produtos, key=lambda produto:
produto["preço"])

    for produto in produtos_ordenados:
        print(f"Nome: {produto['nome']}|Preço:
R${produto['preço']:.2f}|Quantidade:
{produto['quantidade']}|Categoria: {produto['categoria']}")

    #Criada a função que será responsável por remover um dicionário da
lista de produtos, ou seja, um produto. Aqui no começo fazemos uma
tratativa de erro para caso o nome que o usuário queira remover seja
nada, ele digitou, deu um missclick e foi nada pro sistema, então
ele retorna uma mensagem de texto falando que o nome não pode ser
vazio. Depois disso, uma repetição For In onde usamos If para caso o
nome que o usuário queira remover seja igual ao nome do produto na
lista. Caso ocorra, ele remove com um .remove() e retorna para o
menu. Se não, ele só vai printar que o produto não foi encontrado

def remover_produto():
    print("\n=====REMOVER PRODUTO=====")
    try:
        nome_remove = input("Digite o nome do produto que deseja
remover: ").strip().lower().capitalize()
        if nome_remove == "":
            raise ValueError("O nome não pode ser vazio")

        for produto in produtos:
            if produto["nome"] == nome_remove:
                produtos.remove(produto) # remove o dicionário da
lista de produtos (produto.remove não existe, dict não tem esse
método)

                print(f"Produto '{nome_remove}' removido com
sucesso!")

                return
        print("Produto não encontrado. Nada foi removido.")
    except ValueError as erro:
        print(f"Erro: {erro}")

def categoria_maior_valor():
    print("\n=====CATEGORIA COM MAIOR VALOR EM ESTOQUE=====")
    if len(produtos) == 0:
        print("Nenhum produto cadastrado.")
        return

    #Dicionário auxiliar: chave = categoria, valor = soma do (preço
* quantidade)
    #de todos os produtos daquela categoria
    valores_por_categoria = {}

```

```

        for produto in produtos:
            categoria = produto["categoria"]
            valor_produto = produto["preço"] * produto["quantidade"]
            #.get(categoria, 0) retorna 0 se a categoria ainda não
            existir no dicionário,
            #evitando a necessidade de um if/else para inicializar a
            chave
            valores_por_categoria[categoria] =
valores_por_categoria.get(categoria, 0) + valor_produto

        categoria_top = None
        maior_valor = -1
        for categoria, valor in valores_por_categoria.items():
            if valor > maior_valor:
                maior_valor = valor
                categoria_top = categoria
        print(f"Categoria com maior valor total em estoque:
{categoria_top} (R${maior_valor:.2f})")

def menu():
    while True:
        print("\n=====KABUM=====")
        print("1 - Listar produtos\n2 - Cadastrar produtos\n3 -
Buscar produtos\n4 - exibir análise de estoque\n5 - Ordenar
Produtos\n6 - Remover Produtos\n7 - Categoria de Maior Valor\n8 -
Sair")

        op = input("\n-> ").strip()
        match op:
            case "1":
                listar_produto()
            case "2":
                cadastrar_produto()
            case "3":
                buscar_produto()
            case "4":
                analisar_estoque()
            case "5":
                ordenar_produto()
            case "6":
                remover_produto()
            case "7":
                categoria_maior_valor()
            case "8":
                print("\nEncerrando sistema...")
                break
            case _:
                print("Comando inválido, digite '1', '2', '3','4',
'5', '6', '7' ou '8'")

    menu()

```